International Cybersecurity and Digital Forensics Academy (ICDFA)

CIP-B101 Basic Computer Skills for Digital Forensics

Lab 1A: Number Systems, ASCII, Timestamps and Data Representation

Student name	Lelah Nde Ide Naomie
Student ID / Registration no.	C11/26/FCDF/17134
Course code	CIP-B101 (WADF201 Practical Laboratories)
Lab	Lab 1A
Date	6 November 2026
Platform used	Kali Linux (rolling) in a virtual machine, Python 3.13.12, xxd 2025-11-26
Submission file	CIP-B101_Lab1A_Lelah_Naomie.pdf

1. Introduction and Tools

In this laboratory I practised interpreting numbers, text encodings, timestamps, hashes and byte order, because misreading any of these can place events at the wrong time or misidentify evidence. I did all the work myself in a Linux terminal on a Kali Linux virtual machine (VirtualBox, user kali@lelahnaomie, Python 3.13.12), using bash, python3, bc, xxd, date, md5sum and sha256sum. I worked out the conversions manually using the methods shown, verified them with commands, took screenshots as evidence, and explained what each result means.

2. Working Folder (Evidence 1)

I opened a terminal on my Kali VM and created the working folder cip-b101-lab1a with mkdir, moved into it with cd and confirmed the location with pwd. Before each set of commands I printed a line with my name and student ID so that every screenshot is traceable to me.

```
mkdir -p ~/cip-b101-lab1a
cd ~/cip-b101-lab1a
pwd
```



Screenshot 1: working folder creation and pwd output (own screenshot, student ID C11/26/FCDF/17134)

3. Task A: Number Systems and Manual Conversion

3.1 Manual working

I converted each value manually. For binary and hexadecimal to decimal, I wrote the weight of each digit and added the results. For decimal to binary and hexadecimal, I divided repeatedly and read the remainders from the bottom up. For binary to hexadecimal, I grouped the bits in fours from the right.

(1) 101010 (binary) to decimal. Weights from the right: 1,2,4,8,16,32.

Bit position (right to left)	5	4	3	2	1	0
Bit	1	0	1	0	1	0
Weight (2^n)	32	16	8	4	2	1
Contribution	32	0	8	0	2	0

32 + 8 + 2 = 42. Answer: 42

(2) AB2 (hex) to decimal. A=10, B=11.

A x 16^2 = 10 x 256 = 2560; B x 16^1 = 11 x 16 = 176; 2 x 16^0 = 2. Sum = 2560 + 176 + 2 = 2738. Answer: 2738

(3) 126 (decimal) to binary by repeated division by 2.

Division	Quotient	Remainder
126 / 2	63	0
63 / 2	31	1
31 / 2	15	1
15 / 2	7	1
7 / 2	3	1
3 / 2	1	1
1 / 2	0	1

Read remainders from bottom to top: 1111110. Check: 64+32+16+8+4+2 = 126. Answer: 1111110

(4) 2738 (decimal) to hexadecimal by repeated division by 16.

Division	Quotient	Remainder
2738 / 16	171	2
171 / 16	10	11 (B)
10 / 16	0	10 (A)

Read remainders from bottom to top: A B 2. Answer: AB2

(5) 101010110010 (binary) to hexadecimal, grouping in fours from the right.

1010 | 1011 | 0010 = A | B | 2. Answer: AB2 (which equals 2738 decimal, consistent with items 2 and 4).

3.2 Validation commands

I then checked every manual answer with bash arithmetic, printf, bc and Python. All outputs matched my manual results. At first bc was not installed on my VM, which I explain in section 9.

```
echo $((2#101010))
echo $((16#AB2))
echo "obase=2;126" | bc
printf "%X\n" 2738
echo "ibase=2;101010110010" | bc     # decimal check of item 5
printf "%X\n" $((2#101010110010))
python3 - <<'PY'
```

```
print(int('101010', 2))
print(int('AB2', 16))
print(bin(126))
print(hex(2738))
PY
```



Screenshot 2: conversion commands and outputs (shell arithmetic, bc, printf, Python) (own screenshot, student ID C11/26/FCDF/17134)

Observed outputs: 42, 2738, 1111110, AB2, AB2; Python printed 42, 2738, 0b1111110, 0xab2. All five results match my manual working in section 3.1.

3.3 Evidence table

Evidence Value	Given Base	Manual Answer	Command Used to Verify	Screenshot Ref.
101010	Binary	42 decimal	echo $((2#101010))	Screenshot 2
AB2	Hexadecimal	2738 decimal	echo $((16#AB2))	Screenshot 2
126	Decimal	1111110 binary	echo "obase=2;126" | bc	Screenshot 2
2738	Decimal	AB2 hexadecimal	printf "%X\n" 2738	Screenshot 2
101010110010	Binary	AB2 hex (2738 decimal)	printf "%X\n" $ ((2#101010110010))	Screenshot 2

4. Task B: ASCII, UTF-8 and Hex Evidence

I created evidence_text.txt containing Hello with echo -n, viewed it in hexadecimal with xxd and xxd -p, and converted the hex back to text with printf and xxd -r -p. I then did the same with my own file, naomie_lab1a.txt, containing Digital Forensics Begins With Bytes., and compared it with a second file made with plain echo, which has an extra 0a byte at the end.

```
echo -n "Hello" > evidence_text.txt
cat evidence_text.txt
xxd evidence_text.txt
xxd -p evidence_text.txt
printf "\x48\x65\x6c\x6c\x6f\n"
printf "48656c6c6f" | xxd -r -p; printf "\n"
```

Observed: xxd shows 4865 6c6c 6f with the ASCII column reading Hello, and xxd -p gives 48656c6c6f. Mapping: H=0x48, e=0x65, l=0x6c, l=0x6c, o=0x6f. The printf "48656c6c6f" | xxd -r -p command converts the plain hex back to the text Hello.



Screenshot 3: xxd and xxd -p output for evidence_text.txt (own screenshot, student ID C11/26/FCDF/17134)

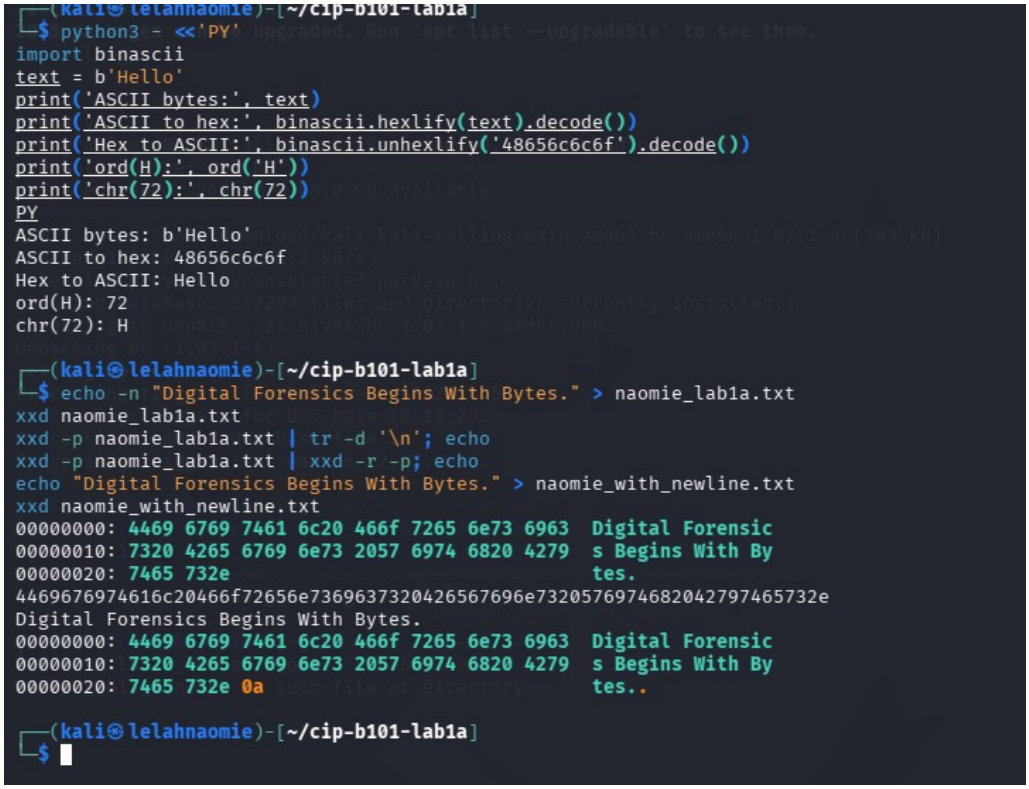
4.1 Student exercise

```
echo -n "Digital Forensics Begins With Bytes." > naomie_lab1a.txt
xxd naomie_lab1a.txt
xxd -p naomie_lab1a.txt | tr -d '\n'; echo
xxd -p naomie_lab1a.txt | xxd -r -p; echo
```

Plain hex of the 36-byte file (created with echo -n, so no trailing 0a):

```
4469676974616c20466f72656e73696373320426567696e7320576974682042797465732e
```

Observed: naomie_lab1a.txt was created with echo -n, so it is 36 bytes with no trailing 0a, and xxd -p | xxd -r -p returned the original sentence. A second file, naomie_with_newline.txt, was created with plain echo; its last byte is 0a (shown at offset 0x24), which makes it 37 bytes.



Screenshot 4: Python ASCII/hex conversion, student file naomie_lab1a.txt, xxd -p, xxd -r -p and the newline file (own screenshot, student ID C11/26/FCDF/17134)

Why 0A appears when echo is used without -n: echo appends a newline character after its output. In ASCII, 0x0A is Line Feed (LF), the Unix end-of-line byte. It is invisible in a terminal, but it is a real byte stored in the file, so xxd shows it as the final byte and the file is one byte longer.

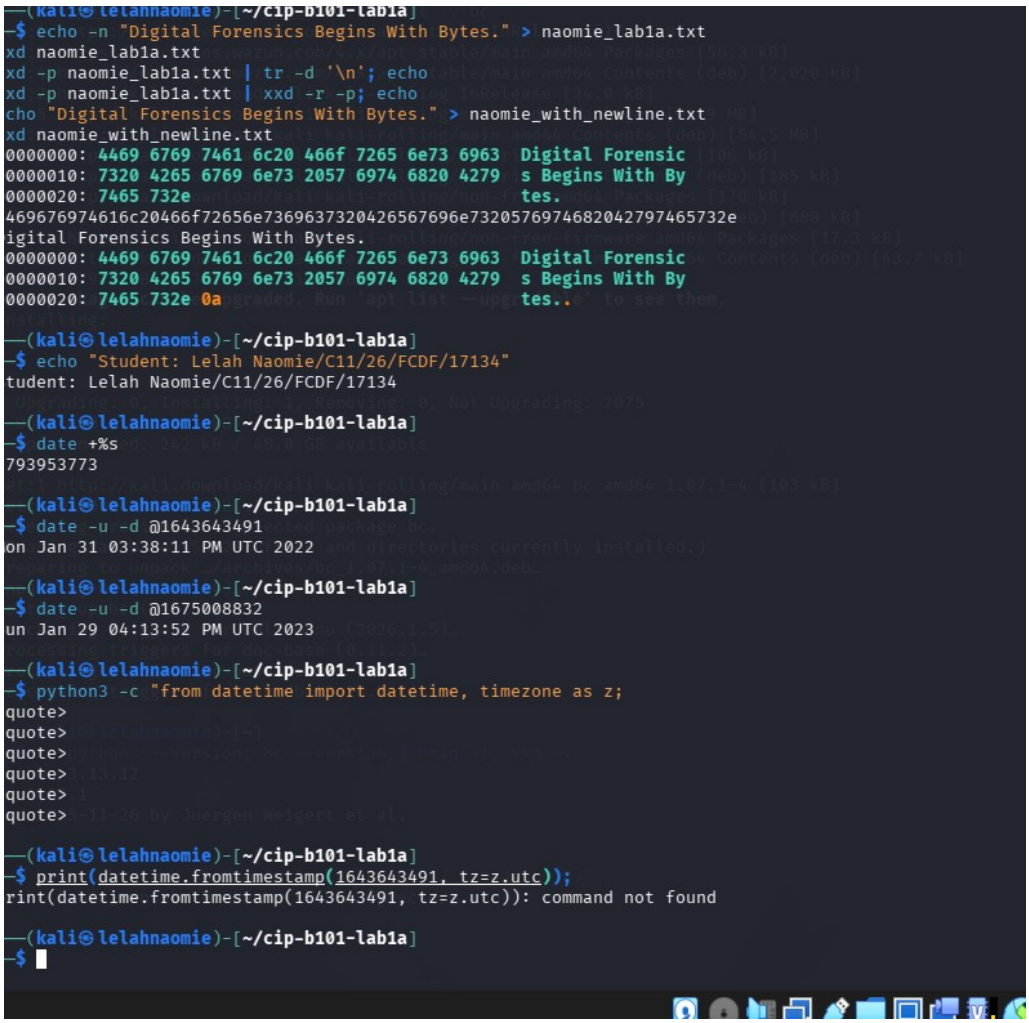
Forensic explanation: an examiner must not assume every hex value is readable text. A byte such as 0x48 is only 'H' if the data is text. The same bytes can be a number, a pointer, part of a multi-byte UTF-8 character, a file signature (for example 89 50 4E 47 for PNG), compressed data or encrypted content, and the latter two look like random bytes. The meaning comes from the context: the file type, the structure being parsed, and the encoding. Forcing a text interpretation on binary data leads to wrong conclusions, and UTF-8 characters outside ASCII use two to four bytes, so a byte-by-byte ASCII reading would also be wrong for them.

5. Task C: Epoch Time and Forensic Timestamps

I converted the two Unix timestamps with date -u and with Python, compared local time with UTC on my VM, and converted the Mac Absolute Time value by adding the 978307200 second difference between the 1970 and 2001 epochs.

```
date +%s
date -u -d @1643643491
```

```
date -u -d @1675008832
python3 - <<'PY'
from datetime import datetime, timezone
print(datetime.fromtimestamp(1643643491, tz=timezone.utc))
print(datetime.fromtimestamp(1675008832, tz=timezone.utc))
PY
```



Screenshot 5: date +%s and date -u -d conversion of both Unix timestamps (own screenshot, student ID C11/26/FCDF/17134)

```
date
date -u
python3 - <<'PY'
from datetime import datetime, timezone
print('Local:', datetime.now())
print('UTC  :', datetime.now(timezone.utc))
PY
```



Screenshot 5b: date, date -u and Python local versus UTC (the invalid option date -U was a typing mistake with no effect) (own screenshot, student ID C11/26/FCDF/17134)

Observed: the Kali date command prints a 12-hour clock with AM/PM, so 03:38:11 PM UTC means 15:38:11 UTC. The Unix timestamps in Screenshot 5 were converted with date -u, so they are UTC, not local time. In Screenshot 5b the VM time zone is EST (UTC-05:00): plain date shows Sat Nov 7 10:44:51 AM EST 2026 while date -u shows 03:45:32 PM UTC, so local time is five hours behind UTC. Python agrees (Local 10:47:38, UTC 15:47:38). This is the VM setting, which may differ from the examiner's real location (WAT, UTC+01:00), which is why every timeline entry must state its zone.

```
python3 - <<'PY'
from datetime import datetime, timezone
unix_epoch = datetime(1970, 1, 1, tzinfo=timezone.utc)
mac_epoch = datetime(2001, 1, 1, tzinfo=timezone.utc)
print(int((mac_epoch - unix_epoch).total_seconds()))
mac_value = 725846400
unix_value = mac_value + 978307200
print(unix_value)
print(datetime.fromtimestamp(unix_value, tz=timezone.utc))
PY
```



Screenshot 6: Mac Absolute Time conversion output (978307200, 1704153600, 2024-01-02 00:00:00 UTC) (own screenshot, student ID C11/26/FCDF/17134)

5.1 Timestamp worksheet

Timestamp Value	Format	Conversion Command	Readable UTC Result	Screenshot Ref.
1643643491	Unix seconds	date -u -d @1643643491	2022-01-31 15:38:11 UTC (Monday)	Screenshot 5
1675008832	Unix seconds	date -u -d @1675008832	2023-01-29 16:13:52 UTC (Sunday)	Screenshot 5
725846400	Mac Absolute Time seconds	python3: datetime.fromtimestamp(725846400 + 978307200, tz=timezone.utc)	2024-01-02 00:00:00 UTC (1704153600 Unix)	Screenshot 6
Current time	Unix seconds	date +%s then date -u -d @<value>	2026-11-06 08:29:33 UTC (from date +%s = 1793953773)	Screenshot 5

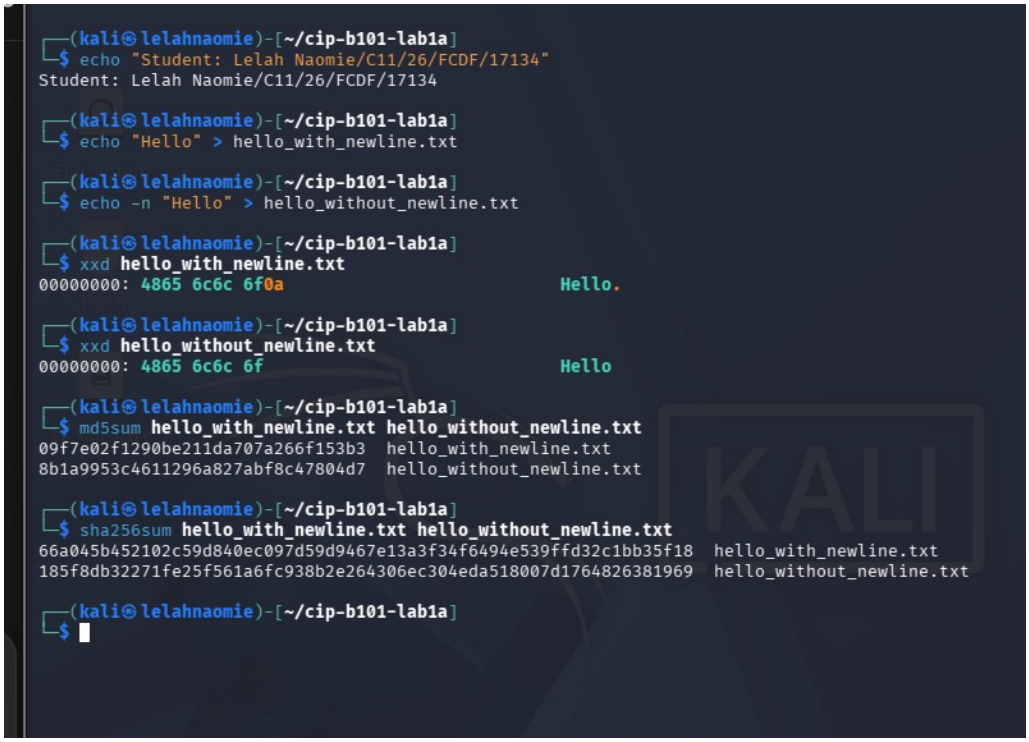
Method for Mac time: Mac Absolute Time counts seconds from 2001-01-01 00:00:00 UTC, Unix from 1970-01-01 00:00:00 UTC; the epochs differ by 978307200 s (31 years incl. 8 leap days: 11323 days x 86400). Unix value = 725846400 + 978307200 = 1704153600.

Why UTC matters: forensic timelines combine sources recorded in different time zones (system logs, browser data, mobile devices). Converting everything to UTC and stating the zone explicitly avoids placing an action hours away from when it actually happened, which could undermine an alibi or a sequence of events. Daylight saving changes and wrong system clocks add further risk, so the examiner also records the device clock offset.

6. Task D: Hashing and the Newline Problem

I created two files with echo and echo -n, compared their bytes with xxd and calculated their md5sum and sha256sum values. I then repeated the exercise with my first name, Naomie.

```
echo "Hello" > hello_with_newline.txt
echo -n "Hello" > hello_without_newline.txt
xxd hello_with_newline.txt
xxd hello_without_newline.txt
md5sum hello_with_newline.txt hello_without_newline.txt
sha256sum hello_with_newline.txt hello_without_newline.txt
```



Screenshot 7: xxd, md5sum and sha256sum for the Hello files (own screenshot, student ID C11/26/FCDF/17134)

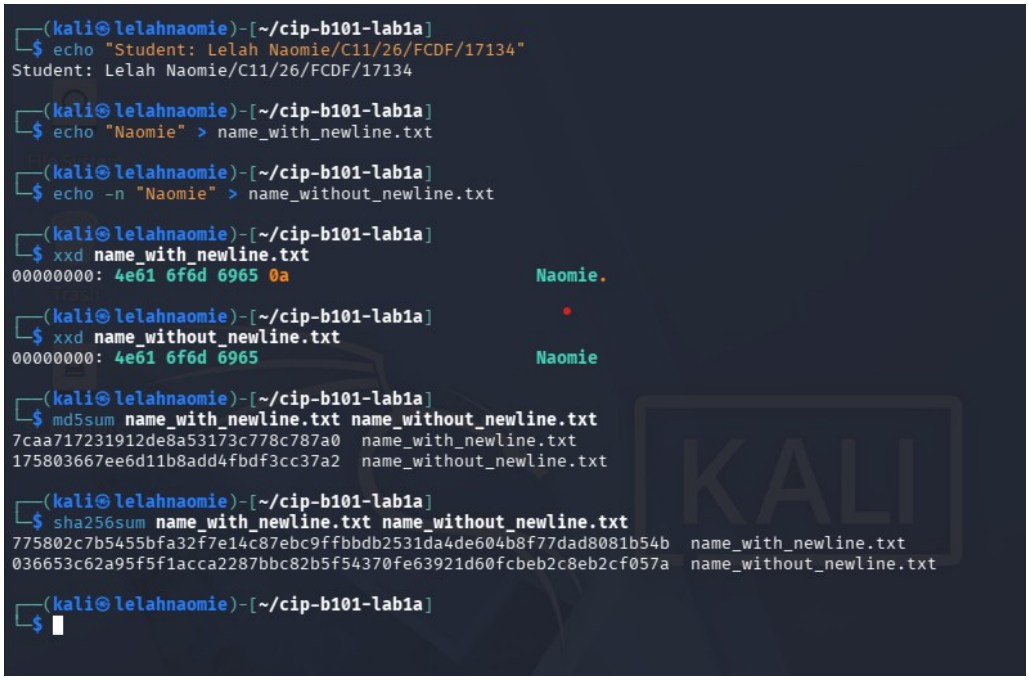
File	Bytes (hex)	MD5	SHA-256
hello_with_newline.txt (echo)	48 65 6c 6c 6f 0a (6 bytes)	09f7e02f1290be211da707a266f153b3	66a045b452102c59d840ec097d59d9467e13a3f34f6494e539ffd32c1bb35f18
hello_without_newline.txt (echo -n)	48 65 6c 6c 6f (5 bytes)	8b1a9953c4611296a827abf8c47804d7	185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969

6.1 Student exercise with my first name

```
echo "Naomie" > name_with_newline.txt
echo -n "Naomie" > name_without_newline.txt
xxd name_with_newline.txt; xxd name_without_newline.txt
md5sum name_*.txt; sha256sum name_*.txt
```

File	Bytes	MD5	SHA-256
name_with_newline.txt	4e 61 6f 6d 69 65 0a (7	7caa717231912de8a53173c	775802c7b5455bfa32f7e14c

(echo "Naomie")	bytes)	778c787a0	87ebc9ffbbdb2531da4de604b8f77dad8081b54b
name_without_newline.txt (echo -n "Naomie")	4e 61 6f 6d 69 65 (6 bytes)	175803667ee6d11b8add4fbdf3cc37a2	036653c62a95f5f1acca2287bbc82b5f54370fe63921d60fcbeb2c8eb2cf057a



Screenshot 7b: xxd, md5sum and sha256sum for the first-name files (hashes match the table above) (own screenshot, student ID C11/26/FCDF/17134)

Why the hashes differ: a cryptographic hash is computed over every byte of the input. The echo version contains one extra byte (0x0A), so the input is different and, because of the avalanche effect, the output changes completely, not by a small amount. The two files look identical on screen, which is exactly why an examiner must compare hashes and bytes rather than visual appearance. In evidence handling this means a single added or removed byte, such as a newline added by an editor, breaks integrity verification.

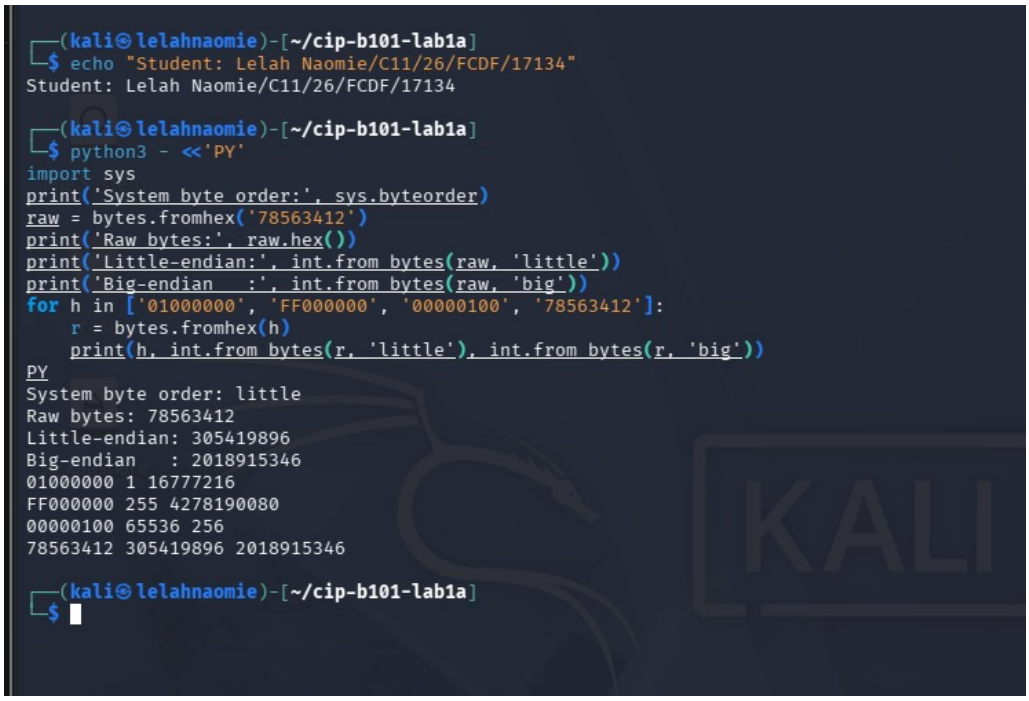
Command to hash exactly the visible characters only: use echo -n (or printf "%s" "text") so no newline is added, e.g. echo -n "Hello" | sha256sum.

7. Task E: Endianness

I used Python to check my system byte order, which is little-endian, and interpreted the four sample byte strings both ways. I first worked out the little-endian value by reversing the bytes, then confirmed it with int.from_bytes.

```
python3 - <<'PY'
import sys
print('System byte order:', sys.byteorder)
raw = bytes.fromhex('78563412')
print('Raw bytes:', raw.hex())
print('Little-endian:', int.from_bytes(raw, 'little'))
```

```
print('Big-endian   :', int.from_bytes(raw, 'big'))
for h in ['01000000','FF000000','00000100','78563412']:
    r = bytes.fromhex(h)
    print(h, int.from_bytes(r,'little'), int.from_bytes(r,'big'))
PY
```



Screenshot 8: endianness Python output (system byte order: little) (own screenshot, student ID C11/26/FCDF/17134)

Raw Hex Bytes	Little-Endian Decimal	Big-Endian Decimal	Command Used	Explanation
01000000	1	16,777,216	int.from_bytes(bytes.fromhex('01000000'),'little'/'big')	LE: first byte is least significant, so value = 0x00000001. BE: first byte is most significant, 0x01000000 = 2^24.
FF000000	255	4,278,190,080	same	LE: 0x000000FF. BE: 0xFF000000 = 255 x 2^24.
00000100	65,536	256	same	LE: bytes reversed give 0x00010000 = 2^16. BE: 0x00000100 = 2^8.
78563412	305,419,896 (0x12345678)	2,018,915,346 (0x78563412)	same	LE reverses byte order to 12 34 56 78; BE reads in the order stored.

Explanation: byte order decides which end of a multi-byte number is stored first. Little-endian (Intel/AMD, and most disk structures such as NTFS and MBR partition fields) stores the least significant byte first; big-endian (network protocols, some file formats) stores the most significant byte first. The same four bytes therefore give different integers depending on the rule used. When an examiner reads a partition's start

sector, a file size, a timestamp or a pointer from a hex view of a disk sector or memory dump, using the wrong order produces a wrong, often absurdly large or small value and a wrong conclusion. For example, in an MBR partition entry the 4-byte sector count is little-endian, so 00 00 10 00 means 0x00100000, not 0x00001000.

8. Task F: Mini Forensic Interpretation Case

I decoded the five evidence items with xxd -r -p and Python and wrote down what each one represents.

```
echo 48656c6c6f204943444641 | xxd -r -p; echo
python3 - <<'PY'
from datetime import datetime, timezone
print(''.join(chr(int(b,2)) for b in '01001000 01101001'.split()))
print(datetime.fromtimestamp(1675008832, tz=timezone.utc))
print(datetime.fromtimestamp(725846400+978307200, tz=timezone.utc))
raw = bytes.fromhex('78563412')
print(int.from_bytes(raw,'little'), int.from_bytes(raw,'big'))
PY
```



Screenshot 9: mini-case interpretation commands and output (own screenshot, student ID C11/26/FCDF/17134)

Evidence Item	Value	Student Interpretation	Command/Screenshot Ref.
EVID-001	48656c6c6f204943444641	Hex-encoded ASCII text: 48 65 6c 6c 6f 20 49 43 44 46 41 = 'Hello ICDFA' (0x20 is a space).	xxd -r -p / Screenshot 9
EVID-002	01001000 01101001	Two 8-bit ASCII bytes: 01001000 = 72 = 'H', 01101001 = 105 = 'i'. Text is 'Hi'.	Python chr(int(b,2)) / Screenshot 9
EVID-003	1675008832	Unix epoch seconds: 2023-01-29 16:13:52 UTC (a Sunday).	date -u -d @1675008832 / Screenshot 9
EVID-004	725846400 (Mac Absolute Time)	Seconds since 2001-01-01 UTC. Add 978307200 to get	Python offset conversion / Screenshot 9

		Unix 1704153600: 2024-01-02 00:00:00 UTC (a Tuesday).	
EVID-005	78563412 as little-endian and big-endian	Little-endian: 0x12345678 = 305,419,896. Big-endian: 0x78563412 = 2,018,915,346. Same bytes, different values because of byte order.	int.from_bytes / Screenshot 9

8.1 Case narrative

After decoding the five items, I found that EVID-001 and EVID-002 are text/content evidence: they decode to 'Hello ICDFA' and 'Hi', showing that readable messages can be hidden in hex or binary notation. EVID-003 and EVID-004 are timestamps useful for timeline analysis: they were converted to 2023-01-29 16:13:52 UTC (Unix epoch method) and 2024-01-02 00:00:00 UTC (Mac Absolute Time converted by adding the 978307200 second offset). Placed in order, EVID-003 occurred about eleven months before EVID-004, and both must be stated in UTC. EVID-005 is a raw 4-byte value whose meaning depends on byte order: 305,419,896 if little-endian and 2,018,915,346 if big-endian. The examiner must establish the correct order from the system architecture or file format specification before reporting any value. Overall, the timestamp items (EVID-003, EVID-004) are the most useful for timelines, and the text items (EVID-001, EVID-002) for content interpretation.

9. Challenges Encountered and How I Resolved Them

Problem I met	What happened	How I resolved it
bc not installed and the VM had no internet	The ping 8.8.8.8 test said Network is unreachable and apt failed with Temporary failure resolving http.kali.org, so I could not install bc.	I corrected the network adapter settings of my VM in VirtualBox so it could reach the internet, installed bc with sudo apt install bc and ran the bc command again.
Files created in the wrong folder	My first xxd screenshot showed the prompt in the home folder (~), so evidence_text.txt was not in my lab folder.	I deleted the stray file with rm, moved into ~/cip-b101-lab1a and repeated the commands so the lab folder shows in the prompt.
printf command not found	I typed printf followed directly by the quoted text with no space, and the shell reported command not found.	I added the missing space after printf and the command printed Hello.
Error when pasting a command	Pasting a command produced a bad pattern error containing the characters ^[[200~.	I typed the command again and kept the working screenshot as evidence.
Python one-liner split across lines	A long python3 -c command broke when pasted, the terminal showed quote> prompts and the rest ran as a separate command that failed.	I pressed Ctrl+C and used the multi-line python3 - <<'PY' format instead, which worked.
Invalid option for date	I typed date -U instead of date -u and the system said invalid option.	I noticed the capital letter was wrong and used the lowercase -u for UTC. The error stays visible in my screenshot.
Time zone and 12-hour clock	Kali showed times such as 03:38:11 PM, and my VM clock was set to EST (UTC-5) instead of my real time zone WAT (UTC+1).	I always converted with date -u and Python in UTC, stated the zone for every result, and noted the difference in the report.

10. Reflection

Number systems and timestamps matter in digital forensics because evidence is stored as bytes, and every conclusion depends on reading those bytes correctly. Hexadecimal is the language of disk sectors, memory and hashes, so an examiner who can convert and verify values by hand and with tools can trust what a tool displays and spot when it is wrong. Timestamps build the timeline of what happened and when, and a mistake between UTC and local time, or between Unix and Mac epochs, can move an event by hours or by thirty years. Hashes prove integrity, and the newline exercise showed that one invisible byte changes the result entirely. Endianness showed me that identical bytes can mean different numbers, so the format and architecture must be known first. In this lab I also learned that small details in a command, such as a missing space before a quoted string or a pasted multi-line command, change the result, so outputs must always be read and checked rather than assumed.

11. Screenshot checklist

No.	Screenshot Evidence	Included?
1	Working folder creation and pwd output	Yes
2	Binary/hex/decimal conversion commands	Yes
3	xxd output for text evidence	Yes
4	Hex-to-ASCII conversion output	Yes
5	Unix timestamp conversion output	Yes
6	Mac Absolute Time conversion output	Yes
7	Hash comparison using md5sum and sha256sum	Yes
8	Endianness Python output	Yes
9	Mini-case evidence interpretation commands	Yes